

1. A Button

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference / steps;
    double turn_angle = 360 / steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();
}
```

```

draw_circle(x,y,70,360);

move_to(x + size * 0.65, y + size * 0.35);
set_font_size(100);
set_pen_color(color::black);
if (symbol == 1)
{
    move_to(x + size * 0.65, y + size * 0.35);
    set_font_size(100);
    write_string(1);
}
else if (symbol == 2)
{
    move_to(x + size * 0.65, y + size * 0.35);
    set_font_size(100);
    write_string(2);
}
else if (symbol == 3)
{
    move_to(x + size * 0.65, y + size * 0.35);
    set_font_size(100);
    write_string(3);
}
else if (symbol == 4)
{
    move_to(x + size * 0.65, y + size * 0.35);
    set_font_size(100);
    write_string(4);
}
else if (symbol == 5)
{
    move_to(x + size * 0.65, y + size * 0.35);
    set_font_size(100);
    write_string(5);
}
else if (symbol == 6)
{
    move_to(x + size * 0.65, y + size * 0.35);
    set_font_size(100);
    write_string(6);
}
else if (symbol == 7)
{
    move_to(x + size * 0.65, y + size * 0.35);

```

```

        set_font_size(100);
        write_string(7);
    }
    else if (symbol == 8)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string(8);
    }
    else if (symbol == 9)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string(9);
    }
    else if (symbol == 0)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string(0);
    }
    else if (symbol == 10)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string("+");
    }
    else if (symbol == 11)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string("-");
    }
    else if (symbol == 12)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_char(L'x');
    }
    else if (symbol == 13)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string(L'↑');
    }

```

```

    }
    else if (symbol == 14)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_char(L'÷');
    }
    else if (symbol == 15)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string("=");
    }
    else if (symbol == 16)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_char(L'©');
    }
    else if (symbol == 17)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_char(L'Σ');
    }
    else if (symbol == 18)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string("M");
    }
    else if (symbol == 19)
    {
        move_to(x + size * 0.65, y + size * 0.35);
        set_font_size(100);
        write_string("C");
    }
}

}

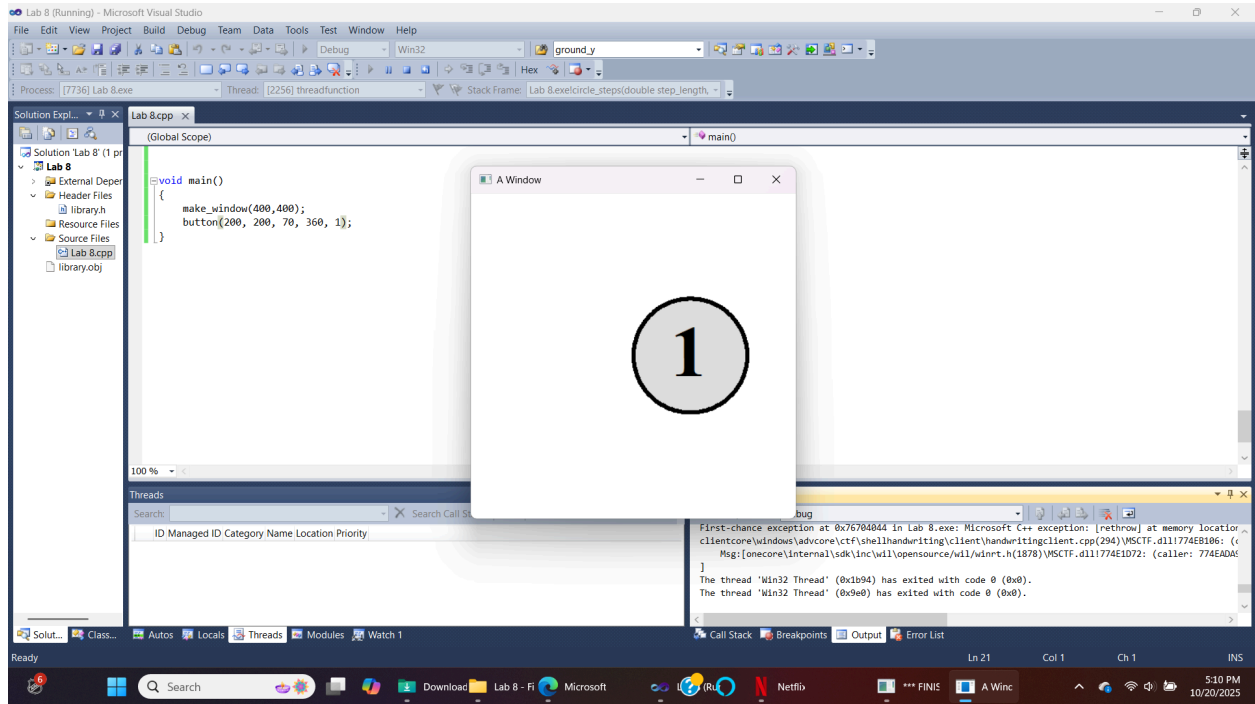
```

```

void main()

```

```
{  
    make_window(400,400);  
    button(200, 200, 70, 360, 1);  
}
```



2. Some Buttons

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol, double font_size)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();

    draw_circle(x,y,size,360);
}
```

```
move_to(x + size * 0.65, y + size * 0.35);
set_font_size(font_size);
set_pen_color(color::black);
if (symbol == 1)
{
    write_string(1);
}
else if (symbol == 2)
{
    write_string(2);
}
else if (symbol == 3)
{
    write_string(3);
}
else if (symbol == 4)
{
    write_string(4);
}
else if (symbol == 5)
{
    write_string(5);
}
else if (symbol == 6)
{
    write_string(6);
}
else if (symbol == 7)
{
    write_string(7);
}
else if (symbol == 8)
{
    write_string(8);
}
else if (symbol == 9)
{
    write_string(9);
}
else if (symbol == 0)
{
    write_string(0);
}
```

```

else if (symbol == 10)
{
    write_string("+");
}
else if (symbol == 11)
{
    move_to(x + size * 0.85, y + size * 0.30);
    write_string("-");
}
else if (symbol == 12)
{
    write_char(L'x');
}
else if (symbol == 13)
{
    move_to(x + size * 0.75, y + size * 0.3);
    write_char(L'↑');
}
else if (symbol == 14)
{
    write_char(L'÷');
}
else if (symbol == 15)
{
    write_string("=");
}
else if (symbol == 16)
{
    write_char(L'©');
}
else if (symbol == 17)
{
    write_char(L'Σ');
}
else if (symbol == 18)
{
    move_to(x + size * 0.53, y + size * 0.35);
    write_string("M");
}
else if (symbol == 19)
{
    write_string("C");
}
}

```

```

void grid(double x, double y, int window_width, int window_height)
{
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;
    double font_size = button_size * 1.2;

    // 1st row
    move_to(x, y);
    button(x, y, button_size, 360, 7, font_size);
    move_to(x + (button_size + spacing_x), y);
    button(x + (button_size + spacing_x), y, button_size, 360, 8, font_size);
    move_to(x + 2 * (button_size + spacing_x), y);
    button(x + 2 * (button_size + spacing_x), y, button_size, 360, 9, font_size);
    move_to(x + 3 * (button_size + spacing_x), y);
    button(x + 3 * (button_size + spacing_x), y, button_size, 360, 10, font_size);
    move_to(x + 4 * (button_size + spacing_x), y);
    button(x + 4 * (button_size + spacing_x), y, button_size, 360, 13, font_size);

    // 2nd row
    move_to(x, y + spacing_y);
    button(x, y + spacing_y, button_size, 360, 4, font_size);
    move_to(x + (button_size + spacing_x), y + spacing_y);
    button(x + (button_size + spacing_x), y + spacing_y, button_size, 360, 5, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + spacing_y);
    button(x + 2 * (button_size + spacing_x), y + spacing_y, button_size, 360, 6, font_size);
    move_to(x + 3 * (button_size + spacing_x), y + spacing_y);
    button(x + 3 * (button_size + spacing_x), y + spacing_y, button_size, 360, 11, font_size);
    move_to(x + 4 * (button_size + spacing_x), y + spacing_y);
    button(x + 4 * (button_size + spacing_x), y + spacing_y, button_size, 360, 16, font_size);

    // 3rd row
    move_to(x, y + 2 * spacing_y);
    button(x, y + 2 * spacing_y, button_size, 360, 1, font_size);
    move_to(x + (button_size + spacing_x), y + 2 * spacing_y);
    button(x + (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 2, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + 2 * spacing_y);

```

```

button(x + 2 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 3, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 12, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 17, font_size);

// 4th row
move_to(x, y + 3 * spacing_y);
button(x, y + 3 * spacing_y, button_size, 360, 19, font_size);
move_to(x + (button_size + spacing_x), y + 3 * spacing_y);
button(x + (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 0, font_size);
move_to(x + 2 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 2 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 15, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 14, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 18, font_size);
}

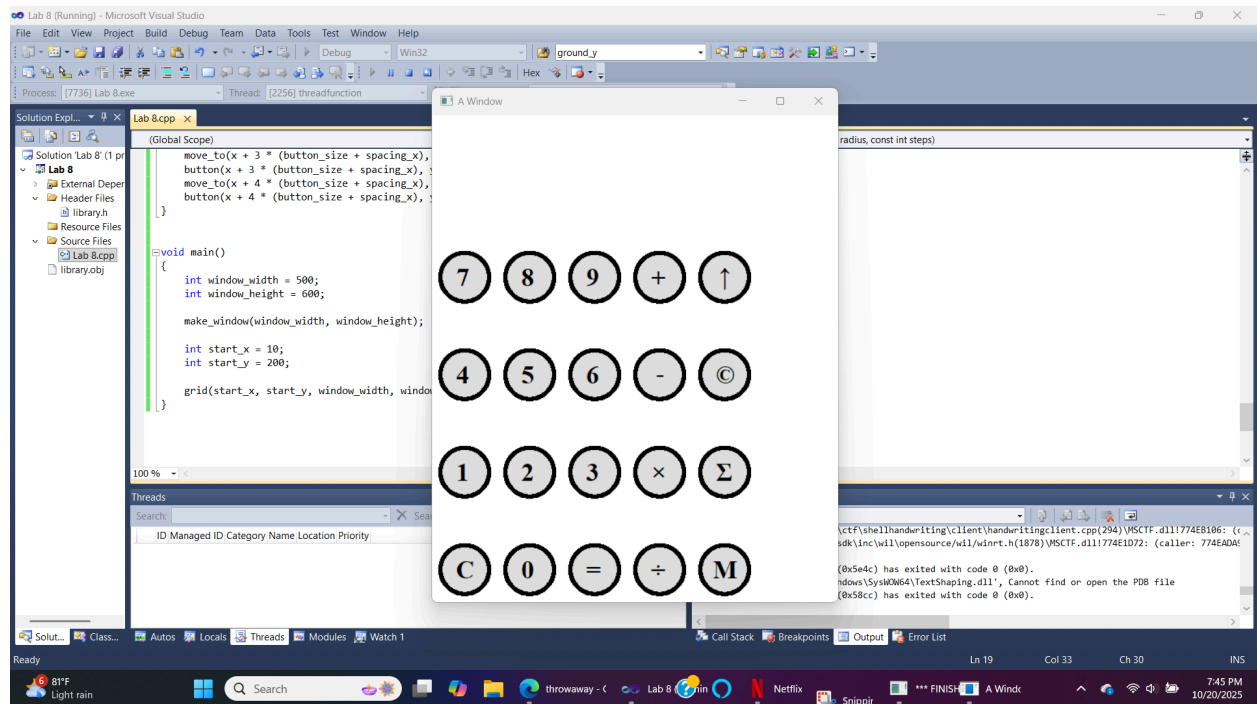
void main()
{
    int window_width = 500;
    int window_height = 600;

    make_window(window_width, window_height);

    int start_x = 10;
    int start_y = 200;

    grid(start_x, start_y, window_width, window_height);
}

```



3. Clicking

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol, double font_size)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();

    draw_circle(x,y,size,360);
}
```

```
move_to(x + size * 0.65, y + size * 0.35);
set_font_size(font_size);
set_pen_color(color::black);
if (symbol == 1)
{
    write_string(1);
}
else if (symbol == 2)
{
    write_string(2);
}
else if (symbol == 3)
{
    write_string(3);
}
else if (symbol == 4)
{
    write_string(4);
}
else if (symbol == 5)
{
    write_string(5);
}
else if (symbol == 6)
{
    write_string(6);
}
else if (symbol == 7)
{
    write_string(7);
}
else if (symbol == 8)
{
    write_string(8);
}
else if (symbol == 9)
{
    write_string(9);
}
else if (symbol == 0)
{
    write_string(0);
}
else if (symbol == 10)
```

```

{
    write_string("+");
}
else if (symbol == 11)
{
    move_to(x + size * 0.85, y + size * 0.30);
    write_string("-");
}
else if (symbol == 12)
{
    write_char(L'x');
}
else if (symbol == 13)
{
    move_to(x + size * 0.75, y + size * 0.3);
    write_char(L'↑');
}
else if (symbol == 14)
{
    write_char(L'÷');
}
else if (symbol == 15)
{
    write_string("=");
}
else if (symbol == 16)
{
    write_char(L'©');
}
else if (symbol == 17)
{
    write_char(L'Σ');
}
else if (symbol == 18)
{
    move_to(x + size * 0.53, y + size * 0.35);
    write_string("M");
}
else if (symbol == 19)
{
    write_string("C");
}
}

```

```

void grid(double x, double y, int window_width, int window_height)
{
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;
    double font_size = button_size * 1.2;

    // 1st row
    move_to(x, y);
    button(x, y, button_size, 360, 7, font_size);
    move_to(x + (button_size + spacing_x), y);
    button(x + (button_size + spacing_x), y, button_size, 360, 8, font_size);
    move_to(x + 2 * (button_size + spacing_x), y);
    button(x + 2 * (button_size + spacing_x), y, button_size, 360, 9, font_size);
    move_to(x + 3 * (button_size + spacing_x), y);
    button(x + 3 * (button_size + spacing_x), y, button_size, 360, 10, font_size);
    move_to(x + 4 * (button_size + spacing_x), y);
    button(x + 4 * (button_size + spacing_x), y, button_size, 360, 13, font_size);

    // 2nd row
    move_to(x, y + spacing_y);
    button(x, y + spacing_y, button_size, 360, 4, font_size);
    move_to(x + (button_size + spacing_x), y + spacing_y);
    button(x + (button_size + spacing_x), y + spacing_y, button_size, 360, 5, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + spacing_y);
    button(x + 2 * (button_size + spacing_x), y + spacing_y, button_size, 360, 6, font_size);
    move_to(x + 3 * (button_size + spacing_x), y + spacing_y);
    button(x + 3 * (button_size + spacing_x), y + spacing_y, button_size, 360, 11, font_size);
    move_to(x + 4 * (button_size + spacing_x), y + spacing_y);
    button(x + 4 * (button_size + spacing_x), y + spacing_y, button_size, 360, 16, font_size);

    // 3rd row
    move_to(x, y + 2 * spacing_y);
    button(x, y + 2 * spacing_y, button_size, 360, 1, font_size);
    move_to(x + (button_size + spacing_x), y + 2 * spacing_y);
    button(x + (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 2, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + 2 * spacing_y);
    button(x + 2 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 3, font_size);
}

```

```

move_to(x + 3 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 12, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 17, font_size);

// 4th row
move_to(x, y + 3 * spacing_y);
button(x, y + 3 * spacing_y, button_size, 360, 19, font_size);
move_to(x + (button_size + spacing_x), y + 3 * spacing_y);
button(x + (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 0, font_size);
move_to(x + 2 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 2 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 15, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 14, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 18, font_size);

    set_pen_color(color::black);

}

void display()
{
    int window_width = 500;
    int window_height = 600;

    double display_height_ratio = 0.15;
    double display_width_ratio = 0.9;
    double display_x = (window_width * (1 - display_width_ratio)) / 2;
    double display_y = 50;

    double display_width = window_width * display_width_ratio;
    double display_height = window_height * display_height_ratio;

    set_pen_color(color::dark_grey);
    fill_rectangle(display_x, display_y, display_width, display_height);
}

int clicks(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y)
{
    wait_for_mouse_click();

```

```
const int x = get_click_x();
const int y = get_click_y();
```

```
int column = (x - start_x) / (button_size + spacing_x) + 1;
int row = (y - start_y) / ((button_size * 2) + spacing_y) + 2.01;
```

```
if (column < 1 || column > 5 || row < 1 || row > 4)
{
    cout << "Clicked outside the calc :(" << endl;
    return clicks(start_x, start_y, button_size, spacing_x, spacing_y);
}
```

```
if (row == 1 && column == 1)
{
    cout << "You clicked row 1 column 1" << endl;
}
if (row == 1 && column == 2)
{
    cout << "You clicked row 1 column 2" << endl;
}
if (row == 1 && column == 3)
{
    cout << "You clicked row 1 column 3" << endl;
}
if (row == 1 && column == 4)
{
    cout << "You clicked row 1 column 4" << endl;
}
if (row == 1 && column == 5)
{
    cout << "You clicked row 1 column 5" << endl;
}
```

```
if (row == 2 && column == 1)
{
    cout << "You clicked row 2 column 1" << endl;
}
if (row == 2 && column == 2)
{
    cout << "You clicked row 2 column 2" << endl;
}
```

```
    }  
    if (row == 2 && column == 3)  
    {  
        cout << "You clicked row 2 column 3" << endl;  
    }  
    if (row == 2 && column == 4)  
    {  
        cout << "You clicked row 2 column 4" << endl;  
    }  
    if (row == 2 && column == 5)  
    {  
        cout << "You clicked row 2 column 5" << endl;  
    }  
}
```

```
if (row == 3 && column == 1)  
{  
    cout << "You clicked row 3 column 1" << endl;  
}  
if (row == 3 && column == 2)  
{  
    cout << "You clicked row 3 column 2" << endl;  
}  
if (row == 3 && column == 3)  
{  
    cout << "You clicked row 3 column 3" << endl;  
}  
if (row == 3 && column == 4)  
{  
    cout << "You clicked row 3 column 4" << endl;  
}  
if (row == 3 && column == 5)  
{  
    cout << "You clicked row 3 column 5" << endl;  
}
```

```
if (row == 4 && column == 1)  
{  
    cout << "You clicked row 4 column 1" << endl;  
}  
if (row == 4 && column == 2)
```

```

        {
            cout << "You clicked row 4 column 2" << endl;
        }
    if (row == 4 && column == 3)
    {
        cout << "You clicked row 4 column 3" << endl;
    }
    if (row == 4 && column == 4)
    {
        cout << "You clicked row 4 column 4" << endl;
    }
    if (row == 4 && column == 5)
    {
        cout << "You clicked row 4 column 5" << endl;
    }
}

```

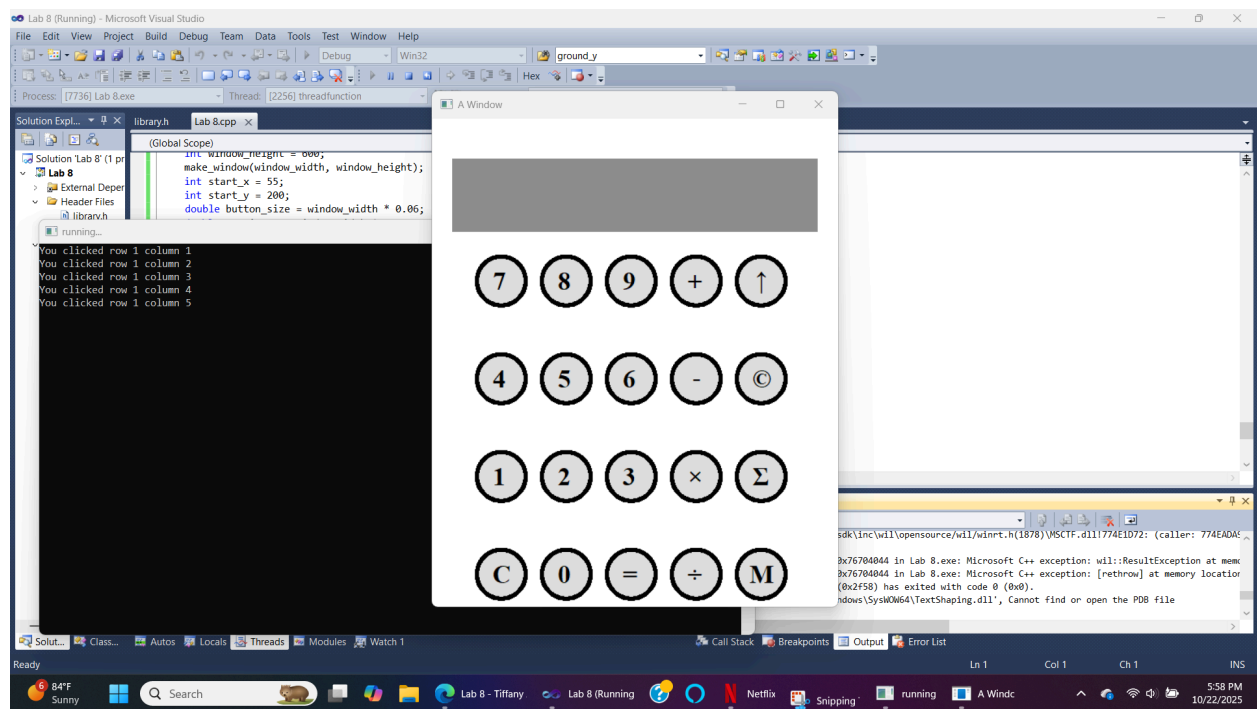
```

void main()
{
    int window_width = 500;
    int window_height = 600;
    make_window(window_width, window_height);
    int start_x = 55;
    int start_y = 200;
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;
    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;

    display();
    grid(start_x, start_y, window_width, window_height);

    while (true)
    {
        const int x = get_click_x();
        const int y = get_click_y();
        int button_logo = clicks(start_x, start_y, button_size, spacing_x, spacing_y);
    }
}

```



4. What did you click on?

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol, double font_size)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();

    draw_circle(x,y,size,360);
}
```

```
move_to(x + size * 0.65, y + size * 0.35);
set_font_size(font_size);
set_pen_color(color::black);
if (symbol == 1)
{
    write_string(1);
}
else if (symbol == 2)
{
    write_string(2);
}
else if (symbol == 3)
{
    write_string(3);
}
else if (symbol == 4)
{
    write_string(4);
}
else if (symbol == 5)
{
    write_string(5);
}
else if (symbol == 6)
{
    write_string(6);
}
else if (symbol == 7)
{
    write_string(7);
}
else if (symbol == 8)
{
    write_string(8);
}
else if (symbol == 9)
{
    write_string(9);
}
else if (symbol == 0)
{
    write_string(0);
}
else if (symbol == 10)
```

```

{
    write_string("+");
}
else if (symbol == 11)
{
    move_to(x + size * 0.85, y + size * 0.30);
    write_string("-");
}
else if (symbol == 12)
{
    write_char(L'x');
}
else if (symbol == 13)
{
    move_to(x + size * 0.75, y + size * 0.3);
    write_char(L'↑');
}
else if (symbol == 14)
{
    write_char(L'÷');
}
else if (symbol == 15)
{
    write_string("=");
}
else if (symbol == 16)
{
    write_char(L'©');
}
else if (symbol == 17)
{
    write_char(L'Σ');
}
else if (symbol == 18)
{
    move_to(x + size * 0.53, y + size * 0.35);
    write_string("M");
}
else if (symbol == 19)
{
    write_string("C");
}
}

```

```

void grid(double x, double y, int window_width, int window_height)
{
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;
    double font_size = button_size * 1.2;

    // 1st row
    move_to(x, y);
    button(x, y, button_size, 360, 7, font_size);
    move_to(x + (button_size + spacing_x), y);
    button(x + (button_size + spacing_x), y, button_size, 360, 8, font_size);
    move_to(x + 2 * (button_size + spacing_x), y);
    button(x + 2 * (button_size + spacing_x), y, button_size, 360, 9, font_size);
    move_to(x + 3 * (button_size + spacing_x), y);
    button(x + 3 * (button_size + spacing_x), y, button_size, 360, 10, font_size);
    move_to(x + 4 * (button_size + spacing_x), y);
    button(x + 4 * (button_size + spacing_x), y, button_size, 360, 13, font_size);

    // 2nd row
    move_to(x, y + spacing_y);
    button(x, y + spacing_y, button_size, 360, 4, font_size);
    move_to(x + (button_size + spacing_x), y + spacing_y);
    button(x + (button_size + spacing_x), y + spacing_y, button_size, 360, 5, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + spacing_y);
    button(x + 2 * (button_size + spacing_x), y + spacing_y, button_size, 360, 6, font_size);
    move_to(x + 3 * (button_size + spacing_x), y + spacing_y);
    button(x + 3 * (button_size + spacing_x), y + spacing_y, button_size, 360, 11, font_size);
    move_to(x + 4 * (button_size + spacing_x), y + spacing_y);
    button(x + 4 * (button_size + spacing_x), y + spacing_y, button_size, 360, 16, font_size);

    // 3rd row
    move_to(x, y + 2 * spacing_y);
    button(x, y + 2 * spacing_y, button_size, 360, 1, font_size);
    move_to(x + (button_size + spacing_x), y + 2 * spacing_y);
    button(x + (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 2, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + 2 * spacing_y);
    button(x + 2 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 3, font_size);
}

```

```

move_to(x + 3 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 12, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 17, font_size);

// 4th row
move_to(x, y + 3 * spacing_y);
button(x, y + 3 * spacing_y, button_size, 360, 19, font_size);
move_to(x + (button_size + spacing_x), y + 3 * spacing_y);
button(x + (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 0, font_size);
move_to(x + 2 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 2 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 15, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 14, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 18, font_size);

    set_pen_color(color::black);

}

void display()
{
    int window_width = 500;
    int window_height = 600;

    double display_height_ratio = 0.15;
    double display_width_ratio = 0.9;
    double display_x = (window_width * (1 - display_width_ratio)) / 2;
    double display_y = 50;

    double display_width = window_width * display_width_ratio;
    double display_height = window_height * display_height_ratio;

    set_pen_color(color::dark_grey);
    fill_rectangle(display_x, display_y, display_width, display_height);
}

int clicks(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y)
{
    wait_for_mouse_click();
    const int x = get_click_x();

```

```

const int y = get_click_y();

int column = (x - start_x) / (button_size + spacing_x) + 1;
int row = (y - start_y) / ((button_size * 2) + spacing_y) + 2.01;

if (column < 1 || column > 5 || row < 1 || row > 4)
{
    cout << "Clicked outside the calc :(" << endl;
    return clicks(start_x, start_y, button_size, spacing_x, spacing_y);
}

if (row == 1 && column == 1)
{
    cout << "You clicked row 1 column 1, aka button 7" << endl;
}
if (row == 1 && column == 2)
{
    cout << "You clicked row 1 column 2, aka button 8" << endl;
}
if (row == 1 && column == 3)
{
    cout << "You clicked row 1 column 3, aka button 9" << endl;
}
if (row == 1 && column == 4)
{
    cout << "You clicked row 1 column 4, aka the add button (symbol 10)" << endl;
}
if (row == 1 && column == 5)
{
    cout << "You clicked row 1 column 5, aka the 'power of' button (symbol 13)" <<
endl;
}

if (row == 2 && column == 1)
{
    cout << "You clicked row 2 column 1, aka button 4" << endl;
}
if (row == 2 && column == 2)
{
    cout << "You clicked row 2 column 2, aka button 5" << endl;
}

```

```

    }
    if (row == 2 && column == 3)
    {
        cout << "You clicked row 2 column 3, aka button 6" << endl;
    }
    if (row == 2 && column == 4)
    {
        cout << "You clicked row 2 column 4, aka the subtract button (symbol 11)" <<
endl;
    }
    if (row == 2 && column == 5)
    {
        cout << "You clicked row 2 column 5, aka the clear memory button (symbol 16)"
<< endl;
    }

```

```

    if (row == 3 && column == 1)
    {
        cout << "You clicked row 3 column 1, aka button 1" << endl;
    }
    if (row == 3 && column == 2)
    {
        cout << "You clicked row 3 column 2, aka button 2" << endl;
    }
    if (row == 3 && column == 3)
    {
        cout << "You clicked row 3 column 3, aka button 3" << endl;
    }
    if (row == 3 && column == 4)
    {
        cout << "You clicked row 3 column 4, aka the multiply button (symbol 12)" <<
endl;
    }
    if (row == 3 && column == 5)
    {
        cout << "You clicked row 3 column 5, aka the sigma button (symbol 17)" << endl;
    }

```

```

    if (row == 4 && column == 1)
    {

```

```

        cout << "You clicked row 4 column 1, aka button C (symbol 19)" << endl;
    }
    if (row == 4 && column == 2)
    {
        cout << "You clicked row 4 column 2, aka button 0" << endl;
    }
    if (row == 4 && column == 3)
    {
        cout << "You clicked row 4 column 3, aka the equal button (symbol 15)" << endl;
    }
    if (row == 4 && column == 4)
    {
        cout << "You clicked row 4 column 4, aka the divide button (symbol 14)" << endl;
    }
    if (row == 4 && column == 5)
    {
        cout << "You clicked row 4 column 5, aka button M (symbol 18)" << endl;
    }
}

```

```

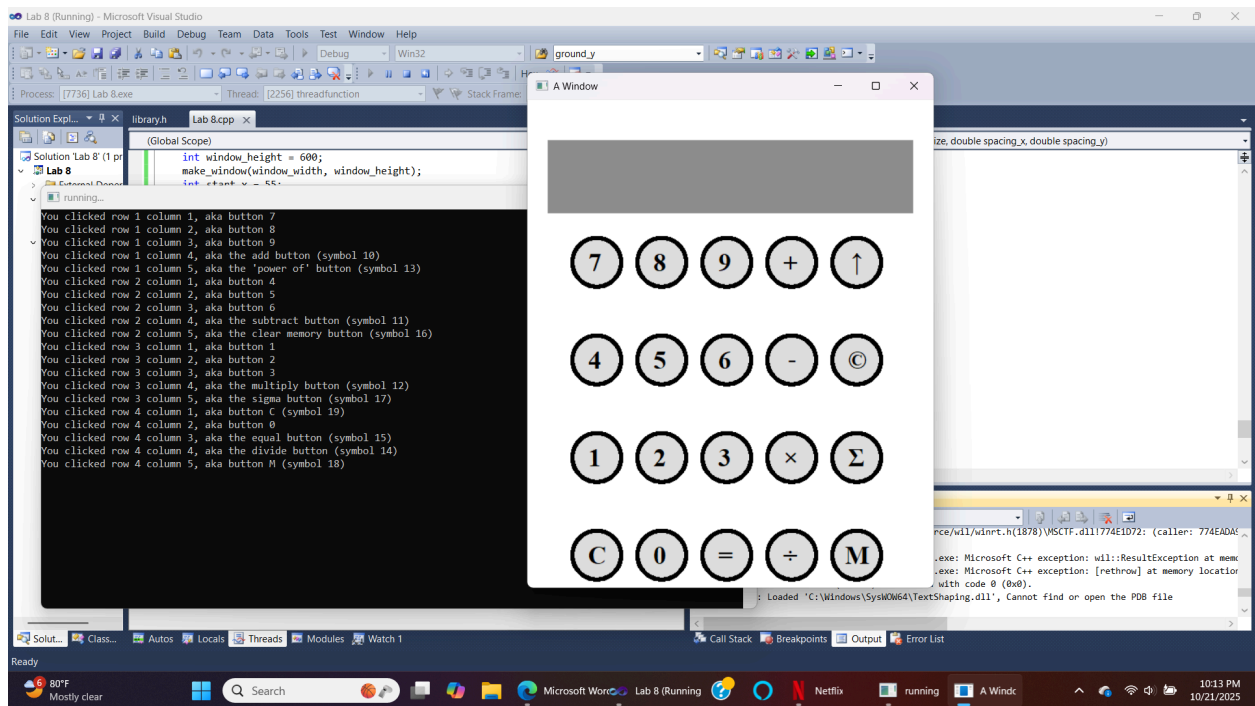
void main()
{
    int window_width = 500;
    int window_height = 600;
    make_window(window_width, window_height);
    int start_x = 55;
    int start_y = 200;
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;
    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;

    display();
    grid(start_x, start_y, window_width, window_height);

    while (true)
    {
        const int x = get_click_x();
        const int y = get_click_y();
        int button_logo = clicks(start_x, start_y, button_size, spacing_x, spacing_y);
    }
}

```

}



5. Entering actual numbers

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol, double font_size)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();

    draw_circle(x,y,size,360);
}
```

```
move_to(x + size * 0.65, y + size * 0.35);
set_font_size(font_size);
set_pen_color(color::black);
if (symbol == 1)
{
    write_string(1);
}
else if (symbol == 2)
{
    write_string(2);
}
else if (symbol == 3)
{
    write_string(3);
}
else if (symbol == 4)
{
    write_string(4);
}
else if (symbol == 5)
{
    write_string(5);
}
else if (symbol == 6)
{
    write_string(6);
}
else if (symbol == 7)
{
    write_string(7);
}
else if (symbol == 8)
{
    write_string(8);
}
else if (symbol == 9)
{
    write_string(9);
}
else if (symbol == 0)
{
    write_string(0);
}
else if (symbol == 10)
```

```

{
    write_string("+");
}
else if (symbol == 11)
{
    move_to(x + size * 0.85, y + size * 0.30);
    write_string("-");
}
else if (symbol == 12)
{
    write_char(L'x');
}
else if (symbol == 13)
{
    move_to(x + size * 0.75, y + size * 0.3);
    write_char(L'↑');
}
else if (symbol == 14)
{
    write_char(L'÷');
}
else if (symbol == 15)
{
    write_string("=");
}
else if (symbol == 16)
{
    write_char(L'©');
}
else if (symbol == 17)
{
    write_char(L'Σ');
}
else if (symbol == 18)
{
    move_to(x + size * 0.53, y + size * 0.35);
    write_string("M");
}
else if (symbol == 19)
{
    write_string("C");
}
}

```

```

void grid(double x, double y, int window_width, int window_height)
{
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;
    double font_size = button_size * 1.2;

    // 1st row
    move_to(x, y);
    button(x, y, button_size, 360, 7, font_size);
    move_to(x + (button_size + spacing_x), y);
    button(x + (button_size + spacing_x), y, button_size, 360, 8, font_size);
    move_to(x + 2 * (button_size + spacing_x), y);
    button(x + 2 * (button_size + spacing_x), y, button_size, 360, 9, font_size);
    move_to(x + 3 * (button_size + spacing_x), y);
    button(x + 3 * (button_size + spacing_x), y, button_size, 360, 10, font_size);
    move_to(x + 4 * (button_size + spacing_x), y);
    button(x + 4 * (button_size + spacing_x), y, button_size, 360, 13, font_size);

    // 2nd row
    move_to(x, y + spacing_y);
    button(x, y + spacing_y, button_size, 360, 4, font_size);
    move_to(x + (button_size + spacing_x), y + spacing_y);
    button(x + (button_size + spacing_x), y + spacing_y, button_size, 360, 5, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + spacing_y);
    button(x + 2 * (button_size + spacing_x), y + spacing_y, button_size, 360, 6, font_size);
    move_to(x + 3 * (button_size + spacing_x), y + spacing_y);
    button(x + 3 * (button_size + spacing_x), y + spacing_y, button_size, 360, 11, font_size);
    move_to(x + 4 * (button_size + spacing_x), y + spacing_y);
    button(x + 4 * (button_size + spacing_x), y + spacing_y, button_size, 360, 16, font_size);

    // 3rd row
    move_to(x, y + 2 * spacing_y);
    button(x, y + 2 * spacing_y, button_size, 360, 1, font_size);
    move_to(x + (button_size + spacing_x), y + 2 * spacing_y);
    button(x + (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 2, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + 2 * spacing_y);
    button(x + 2 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 3, font_size);
}

```

```

move_to(x + 3 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 12, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 17, font_size);

// 4th row
move_to(x, y + 3 * spacing_y);
button(x, y + 3 * spacing_y, button_size, 360, 19, font_size);
move_to(x + (button_size + spacing_x), y + 3 * spacing_y);
button(x + (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 0, font_size);
move_to(x + 2 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 2 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 15, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 14, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 18, font_size);

    set_pen_color(color::black);

}

void display()
{
    int window_width = 500;
    int window_height = 600;

    double display_height_ratio = 0.15;
    double display_width_ratio = 0.9;
    double display_x = (window_width * (1 - display_width_ratio)) / 2;
    double display_y = 50;

    double display_width = window_width * display_width_ratio;
    double display_height = window_height * display_height_ratio;

    set_pen_color(color::dark_grey);
    fill_rectangle(display_x, display_y, display_width, display_height);
}

int click_numbers(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y)
{
    wait_for_mouse_click();
    const int x = get_click_x();

```

```
const int y = get_click_y();
```

```
int column = (x - start_x) / (button_size + spacing_x) + 1;
```

```
int row = (y - start_y) / ((button_size * 2) + spacing_y) + 2.01;
```

```
if (column < 1 || column > 5 || row < 1 || row > 4)
```

```
{
```

```
    cout << "Clicked outside the calc :(" << endl;
```

```
    return click_numbers(start_x, start_y, button_size, spacing_x, spacing_y);
```

```
}
```

```
if (row == 1 && column == 1)
```

```
{
```

```
    cout << "7 ";
```

```
}
```

```
if (row == 1 && column == 2)
```

```
{
```

```
    cout << "8 ";
```

```
}
```

```
if (row == 1 && column == 3)
```

```
{
```

```
    cout << "9 ";
```

```
}
```

```
if (row == 1 && column == 4)
```

```
{
```

```
    cout << "+ ";
```

```
}
```

```
if (row == 1 && column == 5)
```

```
{
```

```
    cout << "You clicked row 1 column 5, aka the 'power of' button (symbol 13)" <<
```

```
endl;
```

```
}
```

```
if (row == 2 && column == 1)
```

```
{
```

```
    cout << "4 ";
```

```
}
```

```
if (row == 2 && column == 2)
```

```
{
```

```

        cout << "5 ";
    }
    if (row == 2 && column == 3)
    {
        cout << "6 ";
    }
    if (row == 2 && column == 4)
    {
        cout << "- ";
    }
    if (row == 2 && column == 5)
    {
        cout << "      MEMORY CLEARED" << endl << endl;
    }

    if (row == 3 && column == 1)
    {
        cout << "1 ";
    }
    if (row == 3 && column == 2)
    {
        cout << "2 ";
    }
    if (row == 3 && column == 3)
    {
        cout << "3 ";
    }
    if (row == 3 && column == 4)
    {
        cout << "You clicked row 3 column 4, aka the multiply button (symbol 12)" <<
endl;
    }
    if (row == 3 && column == 5)
    {
        cout << "You clicked row 3 column 5, aka the sigma button (symbol 17)" << endl;
    }

    if (row == 4 && column == 1)
    {

```

```

        cout << "        Cleared :)" << endl << endl;
    }
    if (row == 4 && column == 2)
    {
        cout << "0 ";
    }
    if (row == 4 && column == 3)
    {
        cout << "= ";
    }
    if (row == 4 && column == 4)
    {
        cout << "You clicked row 4 column 4, aka the divide button (symbol 14)" << endl;
    }
    if (row == 4 && column == 5)
    {
        cout << "Stored!" << endl;
    }
}

```

```

void main()
{
    int window_width = 500;
    int window_height = 600;
    make_window(window_width, window_height);
    int start_x = 55;
    int start_y = 200;
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;
    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;

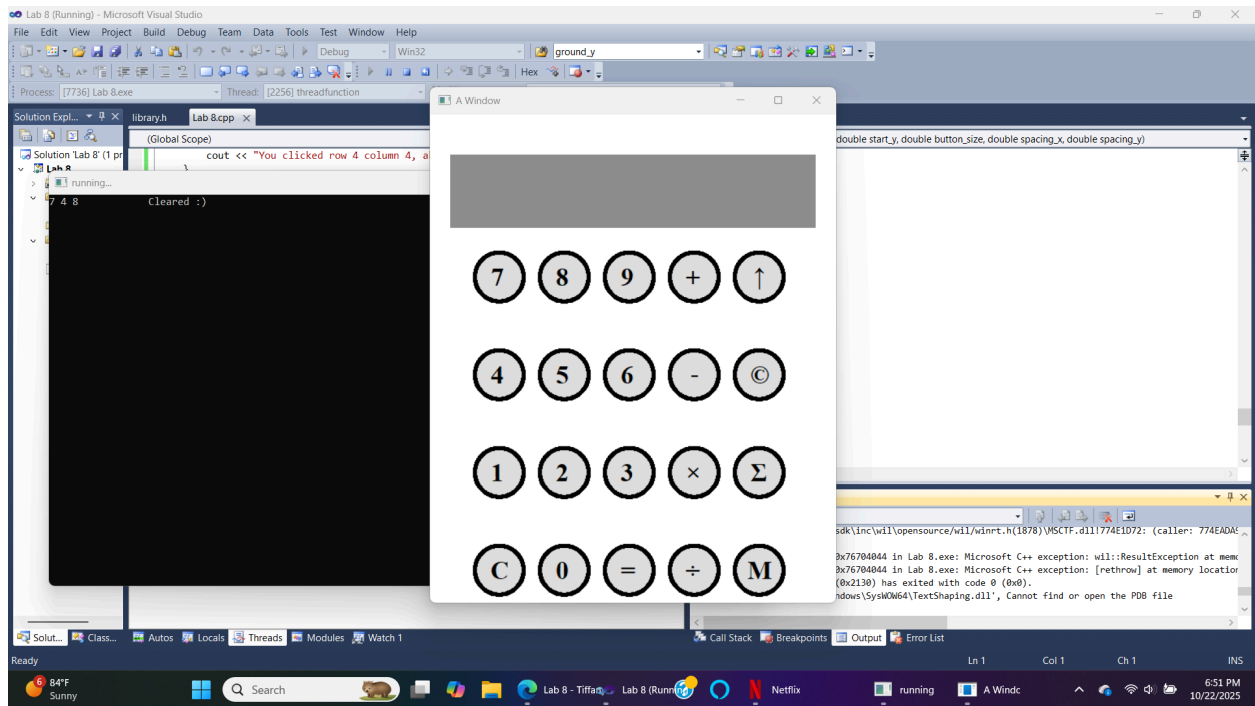
    display();
    grid(start_x, start_y, window_width, window_height);

    while (true)
    {
        const int x = get_click_x();
        const int y = get_click_y();
        int button_logo = click_numbers(start_x, start_y, button_size, spacing_x, spacing_y);
    }
}

```

}

}



6. Calculate!

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol, double font_size)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();

    draw_circle(x,y,size,360);
}
```

```
move_to(x + size * 0.65, y + size * 0.35);
set_font_size(font_size);
set_pen_color(color::black);
if (symbol == 1)
{
    write_string(1);
}
else if (symbol == 2)
{
    write_string(2);
}
else if (symbol == 3)
{
    write_string(3);
}
else if (symbol == 4)
{
    write_string(4);
}
else if (symbol == 5)
{
    write_string(5);
}
else if (symbol == 6)
{
    write_string(6);
}
else if (symbol == 7)
{
    write_string(7);
}
else if (symbol == 8)
{
    write_string(8);
}
else if (symbol == 9)
{
    write_string(9);
}
else if (symbol == 0)
{
    write_string(0);
}
```

```

else if (symbol == 10)
{
    write_string("+");
}
else if (symbol == 11)
{
    move_to(x + size * 0.85, y + size * 0.30);
    write_string("-");
}
else if (symbol == 12)
{
    write_char(L'x');
}
else if (symbol == 13)
{
    move_to(x + size * 0.75, y + size * 0.3);
    write_char(L'↑');
}
else if (symbol == 14)
{
    write_char(L'÷');
}
else if (symbol == 15)
{
    write_string("=");
}
else if (symbol == 16)
{
    write_char(L'©');
}
else if (symbol == 17)
{
    write_char(L'Σ');
}
else if (symbol == 18)
{
    move_to(x + size * 0.53, y + size * 0.35);
    write_string("M");
}
else if (symbol == 19)
{
    write_string("C");
}
}

```

```

void grid(double x, double y, int window_width, int window_height)
{
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;
    double font_size = button_size * 1.2;

    // 1st row
    move_to(x, y);
    button(x, y, button_size, 360, 7, font_size);
    move_to(x + (button_size + spacing_x), y);
    button(x + (button_size + spacing_x), y, button_size, 360, 8, font_size);
    move_to(x + 2 * (button_size + spacing_x), y);
    button(x + 2 * (button_size + spacing_x), y, button_size, 360, 9, font_size);
    move_to(x + 3 * (button_size + spacing_x), y);
    button(x + 3 * (button_size + spacing_x), y, button_size, 360, 10, font_size);
    move_to(x + 4 * (button_size + spacing_x), y);
    button(x + 4 * (button_size + spacing_x), y, button_size, 360, 13, font_size);

    // 2nd row
    move_to(x, y + spacing_y);
    button(x, y + spacing_y, button_size, 360, 4, font_size);
    move_to(x + (button_size + spacing_x), y + spacing_y);
    button(x + (button_size + spacing_x), y + spacing_y, button_size, 360, 5, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + spacing_y);
    button(x + 2 * (button_size + spacing_x), y + spacing_y, button_size, 360, 6, font_size);
    move_to(x + 3 * (button_size + spacing_x), y + spacing_y);
    button(x + 3 * (button_size + spacing_x), y + spacing_y, button_size, 360, 11, font_size);
    move_to(x + 4 * (button_size + spacing_x), y + spacing_y);
    button(x + 4 * (button_size + spacing_x), y + spacing_y, button_size, 360, 16, font_size);

    // 3rd row
    move_to(x, y + 2 * spacing_y);
    button(x, y + 2 * spacing_y, button_size, 360, 1, font_size);
    move_to(x + (button_size + spacing_x), y + 2 * spacing_y);
    button(x + (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 2, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + 2 * spacing_y);

```

```

button(x + 2 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 3, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 12, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 17, font_size);

// 4th row
move_to(x, y + 3 * spacing_y);
button(x, y + 3 * spacing_y, button_size, 360, 19, font_size);
move_to(x + (button_size + spacing_x), y + 3 * spacing_y);
button(x + (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 0, font_size);
move_to(x + 2 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 2 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 15, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 14, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 18, font_size);

    set_pen_color(color::black);

}

void display()
{
    int window_width = 500;
    int window_height = 600;

    double display_height_ratio = 0.15;
    double display_width_ratio = 0.9;
    double display_x = (window_width * (1 - display_width_ratio)) / 2;
    double display_y = 50;

    double display_width = window_width * display_width_ratio;
    double display_height = window_height * display_height_ratio;

    set_pen_color(color::dark_grey);
    fill_rectangle(display_x, display_y, display_width, display_height);
}

int clicks(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y)
{
    wait_for_mouse_click();

```

```

const int x = get_click_x();
const int y = get_click_y();

int column = (x - start_x) / (button_size + spacing_x) + 1;
int row = (y - start_y) / ((button_size * 2) + spacing_y) + 2.01;

if (column < 1 || column > 5 || row < 1 || row > 4)
{
    cout << "Clicked outside the calc :(" << endl;
    return clicks(start_x, start_y, button_size, spacing_x, spacing_y);
}

if (row == 1 && column == 1)
    return 7;
if (row == 1 && column == 2)
    return 8;
if (row == 1 && column == 3)
    return 9;
if (row == 1 && column == 4)
    return 10;
    if (row == 1 && column == 5)
        return 13;

if (row == 2 && column == 1)
    return 4;
if (row == 2 && column == 2)
    return 5;
if (row == 2 && column == 3)
    return 6;
if (row == 2 && column == 4)
    return 11;
    if (row == 2 && column == 5)
        cout << "    MEMORY CLEARED" << endl << endl;

if (row == 3 && column == 1)
    return 1;
if (row == 3 && column == 2)
    return 2;
if (row == 3 && column == 3)
    return 3;
if (row == 3 && column == 4)
    return 12;

```

```

        if (row == 3 && column == 5)
            cout << "Added to Memory!" << endl;

    if (row == 4 && column == 1)
        return 19;
    if (row == 4 && column == 2)
        return 0;
    if (row == 4 && column == 3)
        return 15;
    if (row == 4 && column == 4)
        return 14;
        if (row == 4 && column == 5)
            cout << "Stored!" << endl;
}

int powers(int base_number, int power)
{
    if (power == 0)
        return 1;

    return base_number * powers(base_number, power-1);
}

void calculate(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y, int first_n, int next_n, int symbol)
{
    int button = clicks(start_x, start_y, button_size, spacing_x, spacing_y);

    if (button >= 0 && button <= 9)
    {
        next_n = next_n * 10 + button;
        cout << button;
    }

    else if (button == 10 || button == 11 || button == 12 || button == 13 || button == 14)
    {
        symbol = button;
        first_n = next_n;
        next_n = 0;

        if (button == 10) cout << " + ";
    }
}

```

```

    else if (button == 11) cout << " - ";
    else if (button == 12) cout << " * ";
    else if (button == 13) cout << " ^ ";
    else if (button == 14) cout << " / ";
}

```

```

else if (button == 15)
{
    cout << " = ";

    int result = 0;

    if (symbol == 10) result = first_n + next_n;
    else if (symbol == 11) result = first_n - next_n;
    else if (symbol == 12) result = first_n * next_n;
    else if (symbol == 13) result = powers(first_n, next_n);
    else if (symbol == 14 && next_n != 0) result = first_n / next_n;

    cout << result << endl << endl;

    first_n = 0;
    next_n = 0;
    symbol = 0;
}

```

```

else if (button == 19)
{
    cout << "      Cleared :)" << endl << endl;
    first_n = 0;
    next_n = 0;
    symbol = 0;
}

```

```

    calculate(start_x, start_y, button_size, spacing_x, spacing_y, first_n, next_n, symbol);
}

```

```

void main()
{
    int window_width = 500;

```

```

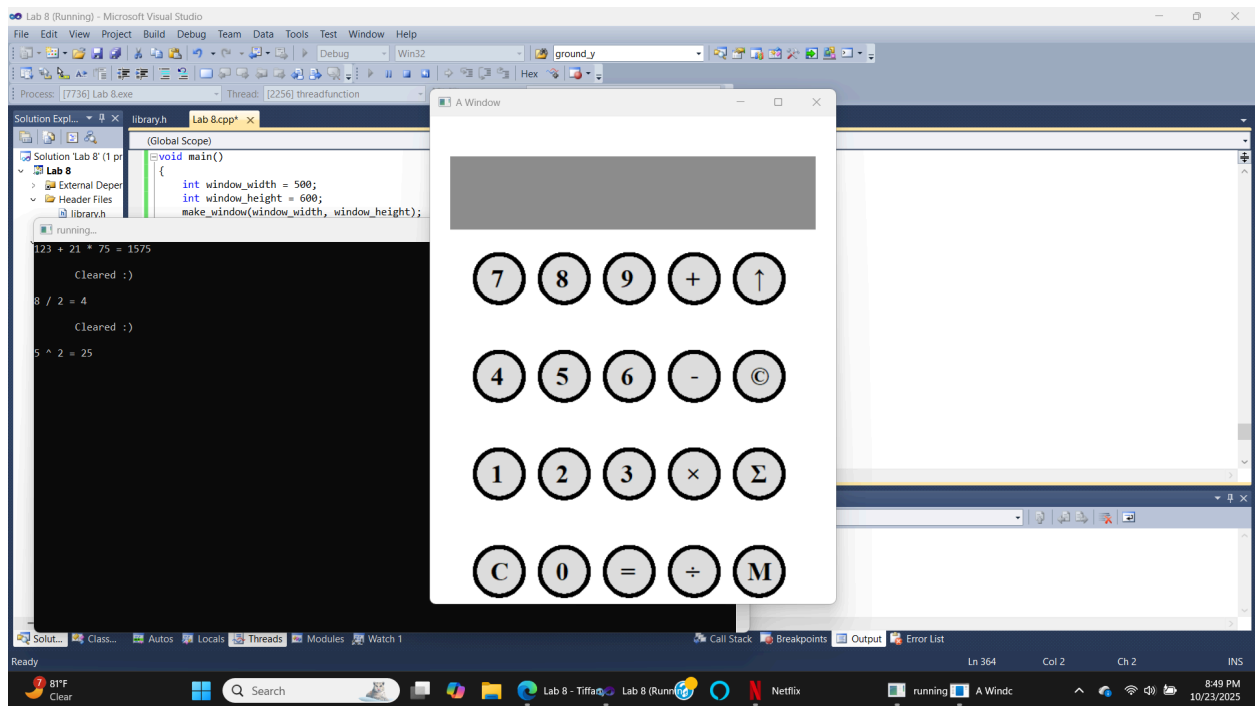
int window_height = 600;
make_window(window_width, window_height);

int start_x = 55;
int start_y = 200;
    double button_size = window_width * 0.06;
double spacing_x = window_width * 0.1;
double spacing_y = window_height * 0.2;

    display();
grid(start_x, start_y, window_width, window_height);
    calculate(start_x, start_y, button_size, spacing_x, spacing_y, 0, 0, 0);

}

```



7. Finally, an actual calculator

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    set_pen_width(5);
    if (steps_left == 0)
    {
        return;
    }
    note_position();
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(int x, int y, double radius, const int steps)
{
    set_pen_color(color::black);
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void button(int x, int y, double size, const int steps, const int symbol, double font_size)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * size;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    start_shape();
    move_to(x,y);
    circle_steps(step_length, turn_angle, steps);
    note_position();
    set_pen_color(color::light_grey);
    fill_shape();

    draw_circle(x,y,size,360);
}
```

```
move_to(x + size * 0.65, y + size * 0.35);
set_font_size(font_size);
set_pen_color(color::black);
if (symbol == 1)
{
    write_string(1);
}
else if (symbol == 2)
{
    write_string(2);
}
else if (symbol == 3)
{
    write_string(3);
}
else if (symbol == 4)
{
    write_string(4);
}
else if (symbol == 5)
{
    write_string(5);
}
else if (symbol == 6)
{
    write_string(6);
}
else if (symbol == 7)
{
    write_string(7);
}
else if (symbol == 8)
{
    write_string(8);
}
else if (symbol == 9)
{
    write_string(9);
}
else if (symbol == 0)
{
    write_string(0);
}
```

```

else if (symbol == 10)
{
    write_string("+");
}
else if (symbol == 11)
{
    move_to(x + size * 0.85, y + size * 0.30);
    write_string("-");
}
else if (symbol == 12)
{
    write_char(L'x');
}
else if (symbol == 13)
{
    move_to(x + size * 0.75, y + size * 0.3);
    write_char(L'↑');
}
else if (symbol == 14)
{
    write_char(L'÷');
}
else if (symbol == 15)
{
    write_string("=");
}
else if (symbol == 16)
{
    write_char(L'©');
}
else if (symbol == 17)
{
    write_char(L'Σ');
}
else if (symbol == 18)
{
    move_to(x + size * 0.53, y + size * 0.35);
    write_string("M");
}
else if (symbol == 19)
{
    write_string("C");
}
}

```

```

void grid(double x, double y, int window_width, int window_height)
{
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    double total_width = 5 * button_size + 4 * spacing_x;
    double total_height = 4 * button_size + 3 * spacing_y;
    double font_size = button_size * 1.2;

    // 1st row
    move_to(x, y);
    button(x, y, button_size, 360, 7, font_size);
    move_to(x + (button_size + spacing_x), y);
    button(x + (button_size + spacing_x), y, button_size, 360, 8, font_size);
    move_to(x + 2 * (button_size + spacing_x), y);
    button(x + 2 * (button_size + spacing_x), y, button_size, 360, 9, font_size);
    move_to(x + 3 * (button_size + spacing_x), y);
    button(x + 3 * (button_size + spacing_x), y, button_size, 360, 10, font_size);
    move_to(x + 4 * (button_size + spacing_x), y);
    button(x + 4 * (button_size + spacing_x), y, button_size, 360, 13, font_size);

    // 2nd row
    move_to(x, y + spacing_y);
    button(x, y + spacing_y, button_size, 360, 4, font_size);
    move_to(x + (button_size + spacing_x), y + spacing_y);
    button(x + (button_size + spacing_x), y + spacing_y, button_size, 360, 5, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + spacing_y);
    button(x + 2 * (button_size + spacing_x), y + spacing_y, button_size, 360, 6, font_size);
    move_to(x + 3 * (button_size + spacing_x), y + spacing_y);
    button(x + 3 * (button_size + spacing_x), y + spacing_y, button_size, 360, 11, font_size);
    move_to(x + 4 * (button_size + spacing_x), y + spacing_y);
    button(x + 4 * (button_size + spacing_x), y + spacing_y, button_size, 360, 16, font_size);

    // 3rd row
    move_to(x, y + 2 * spacing_y);
    button(x, y + 2 * spacing_y, button_size, 360, 1, font_size);
    move_to(x + (button_size + spacing_x), y + 2 * spacing_y);
    button(x + (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 2, font_size);
    move_to(x + 2 * (button_size + spacing_x), y + 2 * spacing_y);

```

```

button(x + 2 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 3, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 12, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 2 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 2 * spacing_y, button_size, 360, 17, font_size);

// 4th row
move_to(x, y + 3 * spacing_y);
button(x, y + 3 * spacing_y, button_size, 360, 19, font_size);
move_to(x + (button_size + spacing_x), y + 3 * spacing_y);
button(x + (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 0, font_size);
move_to(x + 2 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 2 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 15, font_size);
move_to(x + 3 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 3 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 14, font_size);
move_to(x + 4 * (button_size + spacing_x), y + 3 * spacing_y);
button(x + 4 * (button_size + spacing_x), y + 3 * spacing_y, button_size, 360, 18, font_size);

    set_pen_color(color::black);

}

void display(int n)
{
    int window_width = 500;
    int window_height = 600;

    double display_height_ratio = 0.15;
    double display_width_ratio = 0.9;
    double display_x = (window_width * (1 - display_width_ratio)) / 2;
    double display_y = 50;

    double display_width = window_width * display_width_ratio;
    double display_height = window_height * display_height_ratio;

    set_pen_color(color::black);
    fill_rectangle(display_x, display_y, display_width, display_height);

    set_pen_color(color::pink);
    set_font_size(40);
    move_to(display_x + 20, display_y + display_height * 0.7);
    write_string(n);
}

```

```

int clicks(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y)
{
    wait_for_mouse_click();
    const int x = get_click_x();
    const int y = get_click_y();

    int column = (x - start_x) / (button_size + spacing_x) + 1;
    int row = (y - start_y) / ((button_size * 2) + spacing_y) + 2.0005;

    if (column < 1 || column > 5 || row < 1 || row > 4)
    {
        cout << "Clicked outside the calc :(" << endl;
        return clicks(start_x, start_y, button_size, spacing_x, spacing_y);
    }

    if (row == 1 && column == 1)
        return 7;
    if (row == 1 && column == 2)
        return 8;
    if (row == 1 && column == 3)
        return 9;
    if (row == 1 && column == 4)
        return 10;
        if (row == 1 && column == 5)
            return 13;

    if (row == 2 && column == 1)
        return 4;
    if (row == 2 && column == 2)
        return 5;
    if (row == 2 && column == 3)
        return 6;
    if (row == 2 && column == 4)
        return 11;
        if (row == 2 && column == 5)
            cout << "    MEMORY CLEARED" << endl << endl;

    if (row == 3 && column == 1)
        return 1;

```

```

    if (row == 3 && column == 2)
        return 2;
    if (row == 3 && column == 3)
        return 3;
    if (row == 3 && column == 4)
        return 12;
        if (row == 3 && column == 5)
            cout << "Added to Memory!" << endl;

    if (row == 4 && column == 1)
        return 19;
    if (row == 4 && column == 2)
        return 0;
    if (row == 4 && column == 3)
        return 15;
    if (row == 4 && column == 4)
        return 14;
        if (row == 4 && column == 5)
            cout << "Stored!" << endl;
}

int powers(int base_number, int power)
{
    if (power == 0)
        return 1;

    return base_number * powers(base_number, power-1);
}

void calculate(double start_x, double start_y, double button_size, double spacing_x, double
spacing_y, int first_n, int next_n, int symbol)
{
    int button = clicks(start_x, start_y, button_size, spacing_x, spacing_y);

    if (button >= 0 && button <= 9)
    {
        next_n = next_n * 10 + button;
        cout << button;
        display(next_n);
    }
}

```

```

else if (button == 10 || button == 11 || button == 12 || button == 13 || button == 14)
{
    symbol = button;
    first_n = next_n;
    next_n = 0;

    if (button == 10) cout << " + ";
    else if (button == 11) cout << " - ";
    else if (button == 12) cout << " * ";
    else if (button == 13) cout << " ^ ";
    else if (button == 14) cout << " / ";

    display(first_n);
}

```

```

else if (button == 15)
{
    cout << " = ";

    int result = 0;

    if (symbol == 10) result = first_n + next_n;
    else if (symbol == 11) result = first_n - next_n;
    else if (symbol == 12) result = first_n * next_n;
    else if (symbol == 13) result = powers(first_n, next_n);
    else if (symbol == 14 && next_n != 0) result = first_n / next_n;

    cout << result << endl << endl;

    display(result);

    first_n = 0;
    next_n = 0;
    symbol = 0;
}

```

```

else if (button == 19)
{
    cout << endl << endl << "    Cleared :) " << endl << endl;
    first_n = 0;
    next_n = 0;
    symbol = 0;
}

```

```

        display(0);
    }

    calculate(start_x, start_y, button_size, spacing_x, spacing_y, first_n, next_n, symbol);
}

void main()
{
    int window_width = 500;
    int window_height = 600;
    make_window(window_width, window_height);

    int start_x = 55;
    int start_y = 200;
    double button_size = window_width * 0.06;
    double spacing_x = window_width * 0.1;
    double spacing_y = window_height * 0.2;

    display(0);
    grid(start_x, start_y, window_width, window_height);
    calculate(start_x, start_y, button_size, spacing_x, spacing_y, 0, 0, 0);
}

```

[Link to screen recording of working calculator](#)